PrintOnline.et v2.0 — Master Architecture Document

> **Version:** 2.0.0 FINAL
> **Date:** 2026-02-26
> **Status:** APPROVED FOR IMPLEMENTATION
> **Author:** Kidus (Architecture Lead) + Antigravity AI
> **Company:** Pana Promotion
> **Domain:** printonline.et

Table of Contents

1. Executive Summary
2. System Architecture
3. Technology Stack
4. Database Schema
5. Product Catalog Data Model
6. Authentication & Authorization
7. Frontend Architecture
8. Component Architecture & Decomposition
9. CMS Admin Panel
10. Order Lifecycle & Status System
11. Email Integration
12. Error Handling & Resilience
13. Design Principles & Patterns
14. ERP Integration Strategy (Future)
15. Version Roadmap
16. Appendix: Product Catalog Data

---

1. Executive Summary

1.1 What Is PrintOnline.et?

PrintOnline.et is a **first-of-its-kind Ethiopian e-commerce platform** for print product ordering, built for **Pana Promotion**. It is a **standalone web application** — not a module within Pana ERP — that provides end-to-end product ordering, from browsing a catalog of printing products through option selection, design upload, checkout, and order tracking.

1.2 v2.0 Mission

Transform from a static demo (hardcoded mock data, no backend) into a **production-grade, mobile-first e-commerce platform** with:

- Supabase PostgreSQL backend for all data
- Schema-first type generation
- Full authentication with better-auth

- Complete CMS for product/order/customer management (P0)
- Order lifecycle with dual-mode notifications (admin status changes → user account + email)
- Strict error handling with React Error Boundaries + Suspense
- Maximum component decomposition and reusability

1.3 Key Mandates

Mandate	Description
Schema-First	All TypeScript types generated from Supabase schema
Mobile-First	Every view designed and tested mobile-first
ETB Currency	All prices displayed in Ethiopian Birr
Maximum Decomposition	No component file exceeds ~200 lines; P0 priority
Strict Error Handling	Error Boundaries + Suspense at every route segment
TanStack Query Only	All data fetching via TanStack Query hooks
Zod Validation	All forms and API inputs validated with Zod
CMS as P0	Full product/order/customer CMS ships with MVP
Iteration-Ready Schema	DB schema designed as a boilerplate for heavy future iteration

1.4 Deployment Targets

Environment	Provider	Status
Database	Supabase (Frankfurt `eu-central-1`)	☑ Account created
Hosting	Vercel	📝 To configure
Domain	printonline.et	📝 DNS not configured
Email	Ethio Telecom SMTP (order@printonline.et)	📝 To configure

---

2. System Architecture

2.1 High-Level Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                        PRINTONLINE.ET v2.0                           │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   ┌──────────────────────────────────────────────────────────────┐   │
│   │                    NEXT.JS 16 (APP ROUTER)                    │   │
│   │  React 19 • TypeScript 5 • TanStack Query 5 • Zod 3           │   │
│   │  Tailwind v4 • shadcn/ui • Framer Motion                      │   │
│   ├──────────────────────────────────────────────────────────────┤   │
│   │  ROUTE GROUPS                                                 │   │
```

```
│   │  (storefront)  — Public pages, Header/Footer layout          │   │
│   │  (auth)        — Login, Register, Forgot Password            │   │
│   │  (account)     — User dashboard, Order history               │   │
│   │  (cms)         — Admin CMS (ERP-style UI)                    │   │
│   └──────────────────────────┬──────────────────────────────────┘   │
│                              │                                       │
│                ┌─────────────┼─────────────┐                        │
│                │             │             │                        │
│                ▼             ▼             ▼                        │
│      ┌──────────────┐  ┌──────────┐  ┌──────────────────┐          │
│      │ API Routes   │  │better-auth│  │ Supabase Client  │          │
│      │ (Next.js)    │  │ (Auth)   │  │ (SDK)            │          │
│      └──────────────┘  └──────────┘  └──────────────────┘          │
│             │             │                │                         │
│             └─────────────┼────────────────┘                         │
│                           │                                          │
│                           ▼                                          │
│           ┌──────────────────────────────────┐                       │
│           │    SUPABASE (PostgreSQL + Storage)│                      │
│           │  • Categories & Products          │                      │
│           │  • Product Options & Values       │                      │
│           │  • Customer Profiles              │                      │
│           │  • Orders & Order Items           │                      │
│           │  • Storage: product-images (pub)  │                      │
│           │  • Storage: design-uploads (priv) │                      │
│           │  • Row Level Security (RLS)       │                      │
│           └──────────────────────────────────┘                       │
│                                                                      │
│                                                                      │
│           ┌──────────────────────────────────┐                       │
│           │    NODEMAILER (SMTP)              │                      │
│           │  → order@printonline.et           │                      │
│           │  → customer email                 │                      │
│           └──────────────────────────────────┘                       │
│                                                                      │
│   ┌ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ┐  │
│   │   FUTURE v2.5+: PANA ERP INTEGRATION LAYER                  │  │
│   │   • Sync orders → Sales Orders in ERPNext                   │  │
│   │   • Sync inventory levels ← ERP Stock                       │  │
│   │   • CMS embedded as mini-app in ERP UI                      │  │
│   └ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ┘  │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

2.2 Data Flow: Schema-First

```
Supabase Schema (SQL Migrations)
        │
        ▼
  supabase gen types typescript
        │
        ▼
```

```
  types/database.ts (auto-generated interfaces)
        │
        ▼
  Zod schemas (derived from generated types)
        │
        ▼
  TanStack Query hooks (generic, typed)
        │
        ▼
  React Components (type-safe props)
```

2.3 Directory Structure

```
printonline-et/
├── app/
│   ├── (storefront)/                # Public storefront
│   │   ├── layout.tsx               # Header + Footer
│   │   ├── page.tsx                 # Home
│   │   ├── products/[slug]/page.tsx # Product detail
│   │   ├── categories/[slug]/page.tsx # Category (catch-all)
│   │   ├── all-products/page.tsx    # All products A-Z
│   │   ├── search/page.tsx          # Search results
│   │   ├── cart/page.tsx            # Cart
│   │   ├── checkout/page.tsx        # Checkout
│   │   ├── order-summary/page.tsx   # Order summary + T&C
│   │   ├── order-confirmation/page.tsx
│   │   └── contact/page.tsx
│   ├── (auth)/                      # Auth pages
│   │   ├── login/page.tsx
│   │   ├── register/page.tsx
│   │   └── forgot-password/page.tsx
│   ├── (account)/                   # Authenticated user
│   │   ├── layout.tsx
│   │   ├── account/page.tsx         # Dashboard
│   │   └── orders/
│   │       ├── page.tsx             # Order history
│   │       └── [id]/page.tsx        # Order detail + status
│   ├── (cms)/                       # Admin CMS
│   │   ├── layout.tsx               # ERP-style sidebar layout
│   │   └── cms/
│   │       ├── page.tsx             # CMS dashboard
│   │       ├── products/            # Product CRUD
│   │       │   ├── page.tsx         # Product list
│   │       │   ├── new/page.tsx     # Create product
│   │       │   └── [id]/
│   │       │       ├── page.tsx     # Product detail
│   │       │       └── edit/page.tsx # Edit product
│   │       ├── orders/              # Order management
│   │       │   ├── page.tsx         # Order list + status filters
│   │       │   └── [id]/page.tsx    # Order detail + status update
│   │       ├── customers/           # Customer management
```

```
│   │       │   ├── page.tsx
│   │       │   └── [id]/page.tsx
│   │       └── categories/          # Category management
│   │           ├── page.tsx
│   │           └── [id]/page.tsx
│   ├── api/
│   │   ├── auth/[...all]/route.ts   # better-auth handler
│   │   ├── products/route.ts
│   │   ├── orders/route.ts
│   │   ├── upload/route.ts
│   │   └── send-order-email/route.ts
│   ├── globals.css
│   ├── layout.tsx                   # Root layout
│   ├── LayoutClient.tsx             # Client providers
│   ├── not-found.tsx
│   └── error.tsx                    # Global error boundary
│
├── components/
│   ├── ui/                          # shadcn/ui primitives (14 files)
│   ├── shared/                      # Shared reusable components
│   │   ├── ProductCard.tsx
│   │   ├── ProductGrid.tsx
│   │   ├── PriceDisplay.tsx         # ETB formatting
│   │   ├── QuantitySelector.tsx
│   │   ├── SearchBar.tsx
│   │   ├── FileUpload.tsx
│   │   ├── OrderStatusBadge.tsx
│   │   ├── CategoryNav.tsx          # Mega-menu
│   │   ├── CartDrawer.tsx           # Slide-over drawer
│   │   ├── EmptyState.tsx
│   │   ├── LoadingState.tsx
│   │   └── ErrorFallback.tsx
│   ├── home/                        # Home sections (8 files)
│   ├── product/                     # Product detail (decomposed)
│   │   ├── ProductDetailPage.tsx    # Orchestrator (~200 lines)
│   │   ├── ProductGallery.tsx
│   │   ├── ProductInfo.tsx
│   │   ├── ProductForm.tsx
│   │   ├── ProductTabs.tsx
│   │   ├── ProductActions.tsx
│   │   ├── ProductBreadcrumb.tsx
│   │   ├── FormFields.tsx
│   │   ├── ProductFormSchemas.ts
│   │   └── ProductFormTypes.ts
│   ├── category/                    # Category components (5 files)
│   ├── cart/                        # Cart components
│   ├── checkout/                    # Checkout components
│   ├── account/                     # Account components
│   ├── cms/                         # CMS admin components
│   │   ├── layout/
│   │   │   ├── CMSSidebar.tsx
│   │   │   └── CMSHeader.tsx
│   │   ├── products/
│   │   │   ├── ProductForm.tsx      # Create/Edit form
```

```
│   │   │   ├── ProductList.tsx
│   │   │   └── ProductOptionEditor.tsx
│   │   ├── orders/
│   │   │   ├── OrderList.tsx
│   │   │   ├── OrderDetail.tsx
│   │   │   └── OrderStatusEditor.tsx
│   │   └── shared/
│   │       ├── CMSPageHeader.tsx
│   │       ├── CMSDataTable.tsx
│   │       └── CMSConfirmDialog.tsx
│   ├── Header.tsx                  # Public header
│   └── Footer.tsx                  # Public footer
│
├── hooks/
│   ├── data/                       # TanStack Query hooks
│   │   ├── useProducts.ts
│   │   ├── useProduct.ts
│   │   ├── useCategories.ts
│   │   ├── useOrders.ts
│   │   ├── useSearch.ts
│   │   ├── useCustomers.ts
│   │   └── useProductOptions.ts
│   ├── domain/                     # Business logic
│   │   ├── useCartManager.ts
│   │   ├── useCheckout.ts
│   │   └── useOrderStatus.ts
│   └── ui/
│       ├── useDebounce.ts
│       └── useMediaQuery.ts
│
├── lib/
│   ├── supabase/
│   │   ├── client.ts               # Browser client
│   │   ├── server.ts               # Server client (RSC)
│   │   └── admin.ts                # Service role (API routes)
│   ├── auth.ts                     # better-auth config
│   ├── auth-client.ts              # better-auth client
│   ├── query-client.ts             # TanStack Query config
│   ├── utils.ts                    # cn() + helpers
│   ├── currency.ts                 # ETB formatting
│   ├── email.ts                    # Nodemailer transport
│   └── email-template.ts           # HTML email template
│
├── types/
│   ├── database.ts                 # Auto-generated from Supabase
│   └── index.ts                    # Re-exports + domain types
│
├── context/
│   └── CartContext.tsx              # Cart state + DB sync
│
├── supabase/
│   ├── migrations/
│   │   ├── 001_categories.sql
│   │   ├── 002_products.sql
```

```
│   │   ├── 003_product_images.sql
│   │   ├── 004_product_options.sql
│   │   ├── 005_customers.sql
│   │   ├── 006_orders.sql
│   │   ├── 007_rls_policies.sql
│   │   └── 008_functions.sql
│   └── seed/
│       └── products.sql            # Initial catalog data
│
└── docs/
    ├── ARCHITECTURE_V2.md          # This document (in repo)
    ├── client-data/
    │   └── Products list.pdf
    └── pana-erp-docs/              # ERP reference
```

Iteration-Readiness: This structure supports heavy iteration through:

- **Route groups** — New sections (e.g., [(blog)](file:///c:/Users/kidus/Documents/Projects/printonline-et/app/page.tsx#13-27), [(marketing)](file:///c:/Users/kidus/Documents/Projects/printonline-et/app/page.tsx#13-27)) added without touching existing groups
- **Catch-all routes** — New categories added via DB, zero code changes
- `components/shared/` — New shared components slot in immediately
- `hooks/data/` — New query hooks follow the established pattern
- `supabase/migrations/` — Numbered migrations allow schema evolution
- `components/cms/` — CMS modules mirror storefront structure for consistency

---

3. Technology Stack

Layer	Technology	Version	Purpose
Framework	Next.js	16.x (App Router)	Full-stack React framework
Language	TypeScript	5.x (Strict)	Type safety
UI	React	19.x	Component framework
Styling	Tailwind CSS	v4.x	Utility-first CSS
Components	shadcn/ui	Latest (new-york)	Primitive UI
State	TanStack Query	v5.x	Server state + caching
Forms	React Hook Form	v7.x	Form handling
Validation	Zod	v3.x	Runtime validation
Animation	Framer Motion	v11.x	Micro-interactions
Icons	Lucide React	Latest	Iconography
Toasts	Sonner	Latest	Notifications
Theme	next-themes	v0.4.x	Dark/light mode

Layer	Technology	Version	Purpose
Database	Supabase (PostgreSQL)	Latest	Data store
Storage	Supabase Storage	Latest	Images + uploads
Auth	better-auth	v1.3.x	Authentication
Email	Nodemailer	Latest	SMTP
Deployment	Vercel	Latest	Hosting + CDN

---

4. Database Schema

4.1 Entity Relationship Diagram

```
erDiagram
    categories ||--o{ products : has
    products ||--o{ product_images : has
    products ||--o{ product_options : has
    product_options ||--o{ product_option_values : has
    customer_profiles ||--o{ orders : places
    orders ||--o{ order_items : contains
    products ||--o{ order_items : "referenced in"
```

4.2 Complete SQL Migrations

001_categories.sql

```sql
CREATE TABLE categories (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  name TEXT NOT NULL UNIQUE,
  slug TEXT NOT NULL UNIQUE,
  description TEXT,
  image_url TEXT,
  display_order INTEGER DEFAULT 0,
  is_active BOOLEAN DEFAULT true,
  meta_title TEXT,
  meta_description TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_categories_slug ON categories(slug);
CREATE INDEX idx_categories_active ON categories(is_active) WHERE is_active =
true;
```

002_products.sql

```sql
CREATE TABLE products (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  name TEXT NOT NULL,
  slug TEXT NOT NULL UNIQUE,
  category_id UUID REFERENCES categories(id) ON DELETE SET NULL,
  description TEXT,
  short_description TEXT,
  base_price DECIMAL(12,2) NOT NULL DEFAULT 0,
  currency TEXT NOT NULL DEFAULT 'ETB',
  sku TEXT UNIQUE,
  badge TEXT,
  form_type TEXT NOT NULL DEFAULT 'paper',
  is_active BOOLEAN DEFAULT true,
  in_stock BOOLEAN DEFAULT true,
  stock_status TEXT NOT NULL DEFAULT 'in_stock', -- in_stock, low_stock,
out_of_stock, made_to_order
  min_order_quantity INTEGER DEFAULT 1,
  features JSONB DEFAULT '[]'::jsonb,
  specifications JSONB DEFAULT '[]'::jsonb,
  meta_title TEXT,
  meta_description TEXT,
  display_order INTEGER DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_products_slug ON products(slug);
CREATE INDEX idx_products_category ON products(category_id);
CREATE INDEX idx_products_active ON products(is_active) WHERE is_active = true;
CREATE INDEX idx_products_search ON products
  USING gin(to_tsvector('english', name || ' ' || coalesce(description, '')));
```

003_product_images.sql

```sql
CREATE TABLE product_images (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  product_id UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  image_url TEXT NOT NULL,
  alt_text TEXT,
  display_order INTEGER DEFAULT 0,
  is_primary BOOLEAN DEFAULT false,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_product_images_product ON product_images(product_id);
```

004_product_options.sql

```sql
CREATE TABLE product_options (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  product_id UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  option_key TEXT NOT NULL,
  option_label TEXT NOT NULL,
  field_type TEXT NOT NULL DEFAULT 'select',
  is_required BOOLEAN DEFAULT true,
  display_order INTEGER DEFAULT 0,
  description TEXT,
  group_label TEXT,
  depends_on_option TEXT,
  depends_on_value TEXT,
  UNIQUE(product_id, option_key)
);

CREATE TABLE product_option_values (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  option_id UUID NOT NULL REFERENCES product_options(id) ON DELETE CASCADE,
  value TEXT NOT NULL,
  label TEXT NOT NULL,
  price_amount DECIMAL(12,2),
  price_type TEXT DEFAULT 'fixed', -- fixed, percentage, multiplier, override
  group_name TEXT,
  description TEXT,
  display_order INTEGER DEFAULT 0,
  is_default BOOLEAN DEFAULT false,
  is_active BOOLEAN DEFAULT true,
  metadata JSONB DEFAULT '{}'::jsonb
);

CREATE INDEX idx_product_options_product ON product_options(product_id);
CREATE INDEX idx_option_values_option ON product_option_values(option_id);
```

005_customers.sql

```sql
CREATE TABLE customer_profiles (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  auth_user_id TEXT NOT NULL UNIQUE,
  full_name TEXT NOT NULL,
  email TEXT NOT NULL,
  phone TEXT,
  tin_number TEXT,
  company_name TEXT,
  address_line1 TEXT,
  address_line2 TEXT,
  city TEXT DEFAULT 'Addis Ababa',
  sub_city TEXT,
  woreda TEXT,
  country TEXT DEFAULT 'Ethiopia',
  is_active BOOLEAN DEFAULT true,
```

```sql
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_customers_auth ON customer_profiles(auth_user_id);
CREATE INDEX idx_customers_email ON customer_profiles(email);
```

006_orders.sql

```sql
CREATE TABLE orders (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  order_number TEXT NOT NULL UNIQUE,
  customer_id UUID REFERENCES customer_profiles(id),
  status TEXT NOT NULL DEFAULT 'pending',
  status_history JSONB DEFAULT '[]'::jsonb,
  subtotal DECIMAL(12,2) NOT NULL DEFAULT 0,
  tax_amount DECIMAL(12,2) DEFAULT 0,
  delivery_fee DECIMAL(12,2) DEFAULT 0,
  total_amount DECIMAL(12,2) NOT NULL DEFAULT 0,
  currency TEXT NOT NULL DEFAULT 'ETB',
  customer_name TEXT NOT NULL,
  customer_email TEXT NOT NULL,
  customer_phone TEXT,
  customer_tin TEXT,
  delivery_address TEXT,
  delivery_city TEXT,
  delivery_sub_city TEXT,
  special_instructions TEXT,
  internal_notes TEXT,
  terms_accepted BOOLEAN DEFAULT false,
  terms_accepted_at TIMESTAMPTZ,
  confirmation_email_sent BOOLEAN DEFAULT false,
  payment_method TEXT,
  payment_status TEXT DEFAULT 'pending',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE order_items (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  order_id UUID NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
  product_id UUID REFERENCES products(id) ON DELETE SET NULL,
  product_name TEXT NOT NULL,
  product_slug TEXT,
  product_image TEXT,
  category TEXT,
  unit_price DECIMAL(12,2) NOT NULL,
  quantity INTEGER NOT NULL DEFAULT 1,
  line_total DECIMAL(12,2) NOT NULL,
  selected_options JSONB DEFAULT '{}'::jsonb,
  design_file_url TEXT,
```

```sql
  design_file_name TEXT,
  design_file_size INTEGER,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_orders_customer ON orders(customer_id);
CREATE INDEX idx_orders_status ON orders(status);
CREATE INDEX idx_orders_number ON orders(order_number);
CREATE INDEX idx_order_items_order ON order_items(order_id);

-- Order number sequence
CREATE SEQUENCE order_number_seq START 1;

CREATE OR REPLACE FUNCTION generate_order_number()
RETURNS TRIGGER AS $$
BEGIN
  NEW.order_number := 'POL-' || TO_CHAR(NOW(), 'YYYY') || '-'
    || LPAD(NEXTVAL('order_number_seq')::TEXT, 5, '0');
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER set_order_number
  BEFORE INSERT ON orders
  FOR EACH ROW
  WHEN (NEW.order_number IS NULL OR NEW.order_number = '')
  EXECUTE FUNCTION generate_order_number();
```

007_rls_policies.sql

```sql
-- Enable RLS on all tables
ALTER TABLE categories ENABLE ROW LEVEL SECURITY;
ALTER TABLE products ENABLE ROW LEVEL SECURITY;
ALTER TABLE product_images ENABLE ROW LEVEL SECURITY;
ALTER TABLE product_options ENABLE ROW LEVEL SECURITY;
ALTER TABLE product_option_values ENABLE ROW LEVEL SECURITY;
ALTER TABLE customer_profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE order_items ENABLE ROW LEVEL SECURITY;

-- Public read for catalog data
CREATE POLICY "Public read categories" ON categories
  FOR SELECT USING (is_active = true);
CREATE POLICY "Public read products" ON products
  FOR SELECT USING (is_active = true);
CREATE POLICY "Public read product_images" ON product_images
  FOR SELECT USING (true);
CREATE POLICY "Public read product_options" ON product_options
  FOR SELECT USING (true);
CREATE POLICY "Public read product_option_values" ON product_option_values
  FOR SELECT USING (is_active = true);
```

```
-- Service role (API routes) can do everything
-- (handled via service_role key in server-side clients)
```

008_functions.sql

```
-- Auto-update updated_at timestamp
CREATE OR REPLACE FUNCTION update_updated_at()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = NOW();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER update_categories_timestamp BEFORE UPDATE ON categories
  FOR EACH ROW EXECUTE FUNCTION update_updated_at();
CREATE TRIGGER update_products_timestamp BEFORE UPDATE ON products
  FOR EACH ROW EXECUTE FUNCTION update_updated_at();
CREATE TRIGGER update_customer_profiles_timestamp BEFORE UPDATE ON
customer_profiles
  FOR EACH ROW EXECUTE FUNCTION update_updated_at();
CREATE TRIGGER update_orders_timestamp BEFORE UPDATE ON orders
  FOR EACH ROW EXECUTE FUNCTION update_updated_at();
```

4.3 Schema Iteration-Readiness

The schema is designed as a **stable boilerplate** that supports feature additions without migration-breaking changes:

Future Feature	Supported By
Product variants (e.g., Paper → A4/A3)	Add `parent_product_id` foreign key to `products`
Coupons/Discounts	New `coupons` table + `orders.coupon_id`
Payment gateway	`orders.payment_method`, `orders.payment_status` already exist
Reviews/Ratings	New `reviews` table linked to `products` + `customer_profiles`
Wishlist	New `wishlists` table linked to `products` + `customer_profiles`
Multi-currency	`products.currency`, `orders.currency` already exist
Inventory levels	Add `stock_quantity`, `reorder_point` columns to `products`
Shipping zones	New `shipping_zones` table + pricing

Future Feature	Supported By
Audit trail	`orders.status_history` JSONB array already tracks changes
Product bundles	New `product_bundles` table with child items

5. Product Catalog Data Model

5.1 Extracted Product Catalog (from Client PDF)

The client provided **15 product types** with detailed options and ETB pricing:

#	Product	Category	Base Price (ETB)	Key Options
1	Business Cards	Digital Paper Prints	3.50–10.00/card	Size, Print Sides, Paper (250/300gsm), Corners, Lamination
2	Brochures	Digital Paper Prints	40.00–80.00/pc	Type (Tri/Bi/Z-fold), Size (A4/A3)
3	Flyers	Digital Paper Prints	5.00–54.00/pc	Size (DL/A6/A5/A4), Print Sides, Paper (80/150gsm)
4	Saddle-Stitched Booklets	Digital Paper Prints	Varies	Size, Cover (250/300gsm), Lamination, Inside Paper, Page Count
5	Perfect-Bound Booklets	Digital Paper Prints	Varies	Same as above
6	Wire-Bound Booklets	Digital Paper Prints	Varies	+ Binding Alignment (Left/Top/Right)
7	Letterhead	Digital Paper Prints	15.00/pc	Fixed: 80gsm, single-side
8	Premium Gift Bags	Promotional Items	250.00–350.00/pc	Size (A4/A5), Orientation, Handle Color
9	Folders	Digital Paper Prints	350.00–450.00/pc	Print Sides, Lamination, Pocket (1/2)
10	Posters	Digital Paper Prints	54.00–90.00/pc	Paper (150/250gsm), Lamination
11	Envelopes	Digital Paper Prints	25.00–50.00/pc	Size (DL/A5/A4)
12	Paper Sticker Sheets	Vinyl Prints & Wraps	13.00–60.00/pc	Size (A4/A5/A6), Lamination
13	Notebooks	Digital Paper Prints	Varies	Size, Sheets, Filler, Cover Print, Paper, Lamination

#	Product	Category	Base Price (ETB)	Key Options
14	Certificate Paper	Digital Paper Prints	40.00–60.00/pc	Paper Type (White/Textured/Golden Frame)
15	Bookmarks	Digital Paper Prints	10.00–24.00/pc	Lamination, Print Side

5.2 Client Notes (from PDF)

The client also included these requirements in the catalog:

1. ☑ Terms and conditions on final ordering step
2. 📝 Credit card payment options for expats (deferred to v2.5+)
3. ☑ Out-of-stock labeling for promotional items → `stock_status` field
4. ☑ Note writing option for additional order info → `special_instructions`
5. ☑ Cart-based multi-product ordering → Add to cart flow

5.3 Product Image Mapping Strategy

Product images will be mapped using a smart name-matching function:

```
// lib/image-mapper.ts
export function mapProductImage(productName: string, images: string[]): string |
undefined {
  const slug = productName.toLowerCase().replace(/[^a-z0-9]+/g, '-');
  return images.find(img => {
    const imgName = img.toLowerCase().replace(/[^a-z0-9]+/g, '-');
    return imgName.includes(slug) || slug.includes(imgName);
  });
}
```

Product images provided by the client will be uploaded to `product-images/{slug}/` in Supabase Storage. Until real images are provided, current sample images serve as placeholders with the mapping function above.

---

6. Authentication & Authorization

6.1 better-auth Configuration

```
// lib/auth.ts
import { betterAuth } from "better-auth";
import { drizzleAdapter } from "better-auth/adapters/drizzle"; // or pg adapter

export const auth = betterAuth({
  database: { /* Supabase PostgreSQL connection */ },
  emailAndPassword: { enabled: true },
  session: {
```

```
    expiresIn: 60 * 60 * 24 * 7, // 7 days
    updateAge: 60 * 60 * 24,     // 1 day
  },
  user: {
    additionalFields: {
      phone: { type: "string", required: false },
      tin_number: { type: "string", required: false },
      company_name: { type: "string", required: false },
    },
  },
});
```

6.2 Auth Flow

```
Register
  → better-auth creates user (email/password)
  → API route creates customer_profiles row
  → Redirect to account dashboard

Login
  → Session created
  → localStorage cart merged with DB cart
  → Redirect to previous page or dashboard

Checkout (Auth Gate)
  → If not authenticated → redirect to /login?redirect=/checkout
  → If authenticated → proceed to checkout
```

6.3 Role-Based Access

Role	Access
Guest	Browse catalog, add to cart (localStorage), search
Customer	All guest + checkout, order history, profile management
Admin	All customer + CMS panel (/cms/*)

Admin role detection: `customer_profiles.role` field or separate admin table. For MVP, first registered admin is identified by email allowlist in environment variable.

---

7. Frontend Architecture

7.1 Mobile-First Design Mandate

Every component is designed mobile-first with responsive breakpoints:

```
Mobile:  < 640px  (default / sm)
Tablet:  640-1024px (md)
Desktop: > 1024px (lg / xl)
```

Tailwind classes are authored mobile-first: base styles for mobile, `md:` and `lg:` for larger screens.

7.2 Catch-All Category Routes

Replace 6 static category pages with one dynamic route:

```
// app/(storefront)/categories/[slug]/page.tsx
export default async function CategoryPage({ params }: { params: Promise<{ slug:
string }> }) {
  const { slug } = await params;
  // Fetch category + products from Supabase
  // Render using CategoryTemplate component
}
```

7.3 Cart Implementation (Slide-Over Drawer)

```
Header [Cart Icon (count badge)]
    │ Click
    ▼
┌──────────────────────────┐
│  CartDrawer (slide-over) │
│  • Item list with options │
│  • Quantity +/- controls  │
│  • Remove item            │
│  • Subtotal              │
│  • [View Cart] button    │
│  • [Checkout] button     │
└──────────────────────────┘
```

Guest users: localStorage. Authenticated users: DB-persisted + localStorage merge on login.

7.4 ETB Currency Display

```
// lib/currency.ts
export function formatETB(amount: number): string {
  return new Intl.NumberFormat('en-ET', {
    style: 'currency',
    currency: 'ETB',
    minimumFractionDigits: 2,
  }).format(amount);
}
```

All price displays use `<PriceDisplay amount={price} />` component wrapping `formatETB()`.

7.5 Global Search

- Search bar in Header with debounced input
- PostgreSQL full-text search via `to_tsvector` index
- Results page at `/search?q=...`
- Search suggestions in dropdown as user types

7.6 Dark Mode

The current OKLCH theme system is solid. The audit will:

1. Scan all components for hardcoded colors (`bg-white`, `text-gray-*`, `border-gray-*`)
2. Replace with theme tokens (`bg-card`, `text-foreground`, `border-border`)
3. Test every page in both modes
4. Ensure the CMS panel also respects the theme

---

8. Component Architecture & Decomposition

8.1 Decomposition Rules (P0)

Rule	Target
Max file length	~200 lines per component file
Single responsibility	Each component does one thing
Props interface	Explicitly typed, Zod-validated where applicable
No inline data	All data from hooks or props
Theme-aware	Use `bg-card`, never `bg-white`
Error boundary	Every route segment has one

8.2 ProductDetailPage Decomposition

Before: 1 file, 935 lines.
After: 8+ files, ~100-200 lines each.

```
ProductDetailPage.tsx (orchestrator, ~150 lines)
  ├── ProductBreadcrumb.tsx
  ├── ProductGallery.tsx      (image carousel + zoom)
  ├── ProductInfo.tsx          (name, price, badge, stock status)
  ├── ProductForm.tsx          (schema-driven options form)
  │     └── FormFields.tsx     (field renderers)
  ├── ProductActions.tsx       (add to cart, wishlist, share)
  ├── ProductTabs.tsx          (description, specs, reviews)
  └── RelatedProducts.tsx      (products from same category)
```

8.3 Shared Component Library

Every shared component follows this pattern:

```
// components/shared/ProductCard.tsx
interface ProductCardProps {
  product: Product;
  variant?: 'grid' | 'list' | 'compact';
  showBadge?: boolean;
  showPrice?: boolean;
}

export function ProductCard({ product, variant = 'grid', ...props }:
ProductCardProps) {
  // ~80 lines max
}
```

---

9. CMS Admin Panel

9.1 CMS Architecture (P0)

The CMS lives at `/(cms)/cms/` and provides full CRUD for:

Module	Features
Products	List, create, edit, delete, manage options/values, images, stock status
Orders	List with status filters, detail view, status update (triggers email + user notification)
Customers	List, view details, order history per customer
Categories	List, create, edit, reorder, toggle active

9.2 CMS UI Design

The CMS UI will match Pana ERP's aesthetic:

- ERP-style sidebar navigation
- Same shadcn/ui component library
- Same OKLCH theme system (light/dark)
- PageHeader → DataTable → Detail/Form pattern (from ERP MODULE_CREATION_WORKFLOW)
- Premium "Big Tech" aesthetic with micro-animations

9.3 Product Management (CMS)

The product form in CMS allows customization of **every data point**:

- Product name, slug, description, short description
- Category assignment
- Base price (ETB)

- SKU, badge, form type
- Stock status (in_stock, low_stock, out_of_stock, made_to_order)
- Features (JSON editor)
- Specifications (JSON editor)
- Product images (upload, reorder, set primary)
- **Product options** (add/edit/delete option groups)
- **Option values** (add/edit/delete values with pricing rules)
- SEO fields (meta title, description)
- Active/inactive toggle

9.4 ERP Embedding Strategy (Future v2.5+)

The CMS will be **plugged into the Pana ERP repo as a foreign mini-app**:

```
pana-erp/
├── app/
│   ├── crm/                    # CRM Module
│   ├── sales/                  # Sales Module
│   ├── manufacturing/          # Manufacturing Module
│   └── webapp/                 # PrintOnline CMS (iframe/micro-frontend)
│       └── page.tsx            # Embeds PrintOnline CMS UI
│
│   # The CMS is served from printonline.et/cms
│   # But displayed within the ERP's sidebar navigation
│   # Using iframe or Module Federation
```

The CMS will use the **exact same UI patterns** as the ERP (factory pattern, PageHeader, DataTable, InfoCard) so it looks native. The data remains in Supabase — the ERP simply provides an embedding frame.

---

10. Order Lifecycle & Status System

10.1 Status Flow

```
pending → confirmed → processing → ready → delivered → completed
   │
   └──────────────→ cancelled
```

10.2 Dual-Mode Notification

When admin changes order status in CMS:

1. **Database:** `orders.status` updated + entry added to `orders.status_history` JSONB
2. **User Account:** Status reflected when user visits `/orders/[id]` (real-time via query invalidation)
3. **Email:** Order status update email sent to customer (non-blocking)

```
// status_history JSONB structure
[
  { "status": "pending", "timestamp": "2026-02-26T14:30:00Z", "note": "Order
placed" },
  { "status": "confirmed", "timestamp": "2026-02-26T15:00:00Z", "by":
"admin@pana.com" },
  { "status": "processing", "timestamp": "2026-02-27T09:00:00Z", "by":
"admin@pana.com" }
]
```

---

11. Email Integration

11.1 Email Events

Event	Recipient	Template
Order placed	order@printonline.et + customer	Full order details
Order status change	Customer	Status update with tracking
Welcome email	Customer	After registration

11.2 Implementation

As documented in the `/email-integration` workflow:

- Nodemailer with Ethio Telecom SMTP (mail.printonline.et:465 SSL)
- Rich HTML templates with inline CSS (email-safe)
- Fire-and-forget (order completion not blocked by email failure)
- Server-side only (credentials in API routes)

---

12. Error Handling & Resilience

12.1 React Error Boundaries

```
app/
├── error.tsx              # Root error boundary
├── (storefront)/
│   ├── error.tsx          # Storefront error boundary
│   ├── products/[slug]/
│   │   ├── error.tsx      # Product page error boundary
│   │   └── loading.tsx    # Product page Suspense fallback
│   ...
├── (cms)/
│   ├── error.tsx          # CMS error boundary
│   ...
```

12.2 Error Handling Strategy

Layer	Strategy
API Routes	try/catch → structured JSON error response `{ success: false, error, details }`
TanStack Query	`onError` callbacks → Sonner toast notifications
Forms	Zod validation → field-level error display
Network	Retry (1x for queries, 0 for mutations)
Auth	Redirect to login on 401, toast on 403
React	Error Boundaries at every route segment + global fallback

12.3 Suspense Integration

```
// app/(storefront)/products/[slug]/loading.tsx
export default function ProductLoading() {
  return <ProductSkeleton />;
}

// app/(storefront)/products/[slug]/error.tsx
'use client';
export default function ProductError({ error, reset }: { error: Error; reset: ()
=> void }) {
  return <ErrorFallback error={error} onReset={reset} />;
}
```

---

13. Design Principles & Patterns

13.1 Code Patterns (Enforced)

Pattern	Implementation
Schema-First	DB → generated types → Zod → hooks → components
TanStack Query Only	No raw fetch, no useState for server data
Zod All Inputs	Every form, every API input validated
Theme-Aware Only	`bg-card` never `bg-white`, `text-foreground` never `text-black`
Mobile-First	Base CSS = mobile, `md:` / `lg:` for larger
Error Boundary Every Route	No unhandled crashes
ETB Currency	`<PriceDisplay>` component for all prices

13.2 Anti-Patterns (Forbidden)

```
// ❌ NEVER: Manual fetch
const res = await fetch('/api/products');

// ☑ ALWAYS: TanStack Query
const { data } = useProducts();

// ❌ NEVER: Hardcoded colors
className="bg-white text-black"

// ☑ ALWAYS: Theme tokens
className="bg-card text-foreground"

// ❌ NEVER: Inline prices
<span>$45.99</span>

// ☑ ALWAYS: PriceDisplay component
<PriceDisplay amount={product.base_price} />

// ❌ NEVER: Untyped data
const data: any = await supabase.from('products').select();

// ☑ ALWAYS: Generated types
const { data } = await supabase.from('products').select().returns<Product[]>();
```

---

14. ERP Integration Strategy (Future)

14.1 Phased Integration Roadmap

Phase	Version	Strategy
Current	v2.0	Fully standalone Supabase. No ERP dependency.
CMS Embedding	v2.5	CMS embedded as mini-app in Pana ERP sidebar. Data stays in Supabase.
Order Sync	v3.0	Orders sync to ERP Sales Orders via webhook/cron.
Inventory Sync	v3.0	ERP stock levels sync back to Supabase product `stock_status`.
Customer Sync	v3.0	Customer profiles sync to ERP Customer DocType.
Full Integration	v4.0	Supabase as read-cache; ERP as source of truth for inventory/orders.

14.2 ERP-Compatible Schema Design

The Supabase schema uses field naming compatible with ERPNext:

- `customer_name`, `customer_email` (match ERPNext Customer fields)
- `order_number` format (`POL-YYYY-NNNNN`) similar to ERPNext naming
- `status_history` JSONB array mimics ERPNext document versioning
- `product_options` structure maps cleanly to ERPNext Item Variants

14.3 CMS-in-ERP Embedding Options

1. **iframe approach** — ERP sidebar links to `https://printonline.et/cms` via iframe
2. **Module Federation** — Webpack 5 Module Federation to share React components
3. **Reverse proxy** — ERP's Next.js routes proxy CMS routes under `/webapp/`

Recommended: **iframe approach** for v2.5 (simplest, no shared state issues). Upgrade to Module Federation for v3.0+ if tighter integration needed.

---

15. Version Roadmap

Version	Codename	Target	Scope
v2.0 MVP	*Genesis*	Today + Sprint 1	Supabase backend, auth, CMS, order flow, email
v2.1	*Polish*	Week 2	Search, mega-menu, dark mode fix, mobile polish
v2.2	*Payments*	Week 3-4	Payment gateway (Telebirr/CBE Birr), invoice generation
v2.5	*Bridge*	Month 2	CMS embedded in ERP, order sync exploration
v3.0	*Connect*	Quarter 2	Full ERP integration (orders, inventory, customers)
v3.5	*Global*	Quarter 3	International: multi-currency, shipping zones, credit card
v4.0	*Scale*	Quarter 4	Analytics, WhatsApp, advanced promotions, product bundles

---

Appendix: Product Catalog Data

Raw Extracted Data

```
Business Cards:
  Options: Size (8.5x5.5cm / 9x5.4cm), Print Sides (1/2), Paper (250/300gsm),
           Corners (Rounded/Sharp), Lamination (None/Matte/Glossy)
  Min Qty: 50
  Prices (ETB/card): 3.50 - 10.00 depending on options

Brochures:
  Options: Type (Tri/Bi/Z-fold), Size (A4/A3)
  Paper: 150gsm
  Min Qty: 50
  Prices: A4=40, A3=80

Flyers:
  Options: Size (DL/A6/A5/A4), Print Sides (1/2), Paper (80/150gsm)
  Min Qty: 50
  Prices: 5.00 - 54.00

Saddle-Stitched / Perfect-Bound / Wire-Bound Booklets:
```

```
    Options: Size (A4/A5/A6), Cover (250/300gsm), Lamination, Inside Paper
  (80/150gsm),
             Page Count (multiples of 4 for saddle-stitched)
    Wire-bound adds: Binding Alignment (Left/Top/Right), A3 size option

  Letterhead: Fixed 80gsm, single-side, 15.00 ETB/pc, min 50
  Premium Gift Bags: Size (A4/A5), Orientation, Handle Color; A5=250, A4=350
  Folders: Print Sides, Lamination, Pocket (1/2); 350-450 ETB
  Posters: A3, Paper (150/250gsm), Lamination; 54-90 ETB
  Envelopes: Size (DL/A5/A4); 25-50 ETB
  Paper Sticker Sheets: Size (A4/A5/A6), Lamination; 13-60 ETB
  Notebooks: Size, Sheets (25/50/100), Filler, Cover Print, Paper, Lamination
  Certificate Paper: Paper Type (White/Textured/Golden); 40-60 ETB
  Bookmarks: 5x15cm, 300gsm, Lamination, Print Side; 10-24 ETB
```

This is the authoritative master architecture document for PrintOnline.et v2.0.

Maintained and updated throughout the project lifecycle.